AI Slack → Jira Ticket Assistant MVP Design Doc

Overview

Build an AI-powered Slack assistant that can generate Jira tickets directly from Slack thread discussions.

The MVP focuses on:

- Thread-scoped context only
- Explicit user invocation via `@Bot`
- AI-generated Jira ticket drafts
- Human confirmation before ticket creation

The primary goal is to validate whether engineering and operational teams find value in reducing the friction of turning Slack conversations into actionable Jira tickets.

Goals

Primary Goals

- Reduce manual Jira ticket creation effort
- Capture operational context directly from Slack discussions
- Improve ticket quality and completeness
- Fit naturally into existing Slack workflows
- Build a demoable prototype for customer feedback

Non-Goals (For MVP)

The following are intentionally out of scope:

- Autonomous ticket generation
- Channel-wide context reconstruction
- Multi-channel incident correlation
- Embedding/vector search systems
- Automatic issue detection
- Persistent memory systems
- AI auto-assignment logic
- Advanced workflow automation

User Experience

Primary Workflow

A user invokes the bot directly inside a Slack thread.

Example:

```
@TicketBot create Jira ticket from this thread
```

The bot: 1. Reads the Slack thread 2. Extracts relevant operational context 3. Generates a Jira ticket draft 4. Presents the draft to the user 5. Creates the Jira ticket after confirmation

---

UX Flow

Step 1 — User Mentions Bot

Inside a Slack thread:

```
@TicketBot create ticket
```

Optional instructions may also be included:

```
@TicketBot create a high priority bug ticket
```

or:

```
@TicketBot summarize this thread into an incident ticket
```

Step 2 — Bot Fetches Thread Context

The backend retrieves:

- Parent message
- All thread replies
- Message timestamps
- User display names

Messages should remain in chronological order.

Step 3 — Context Cleaning

Before sending to the LLM:

- Remove bot messages if desired
- Remove excessive Slack formatting
- Normalize usernames
- Preserve message ordering
- Keep timestamps optional

Do NOT aggressively summarize before the LLM call in MVP.

Raw conversational context is acceptable.

Step 4 — LLM Ticket Generation

The LLM receives:

- Thread messages
- User instructions from bot mention

The LLM generates structured ticket fields.

Example output:

```
{
  "title": "Payments API latency causing checkout failures",
  "description": "...",
  "priority": "High",
  "labels": ["payments", "production"],
  "summary": "..."
}
```

Step 5 — Draft Preview

The bot responds in Slack with a draft preview.

Example:

```
Proposed Jira Ticket

Title:
Payments API latency causing checkout failures

Priority:
High

Summary:
Customers are experiencing checkout failures due to elevated latency in the
payments API after a recent deployment.

[Create Ticket]
[Cancel]
```

This step is critical for:

- User trust
- Human validation
- Preventing hallucinated tickets
- Avoiding accidental ticket spam

Step 6 — Jira Ticket Creation

After confirmation:

- Create Jira issue via Jira REST API
- Return created issue key + URL

Example:

```
Created Jira Ticket: ENG-142

https://company.atlassian.net/browse/ENG-142
```

MVP Scope Constraints

IMPORTANT

The MVP should ONLY support:

- Explicit bot mentions
- Thread-scoped context

If the bot is invoked outside a thread:

- Respond with an instructional message

Example:

```
Please invoke me inside a Slack thread so I can gather conversation context.
```

This constraint intentionally avoids:

- Channel-wide ambiguity
- Complex context reconstruction
- Noise filtering
- Privacy concerns

Technical Architecture

Components

Slack App

Handles:

- App mentions
- Slack event subscriptions
- Interactive button callbacks

Backend API

Handles:

- Slack event processing
- Thread retrieval
- LLM orchestration

- Jira integration

LLM Service

Handles:

- Ticket extraction
- Summarization
- Structured response generation

Jira Service

Handles:

- Jira API payload creation
- Ticket creation
- Error handling

Slack Integration

Required Slack Features

Bot Token Scopes

Recommended scopes:

- `app_mentions:read`
- `channels:history`
- `channels:read`
- `chat:write`
- `groups:history`
- `groups:read`

Additional scopes may be needed depending on workspace setup.

Event Subscription

Subscribe to:

```
app_mention
```

Slack Event Handling

Incoming Mention

Example payload:

```
{
  "type": "app_mention",
  "channel": "C123",
  "thread_ts": "1717000000.123",
  "text": "<@BOT> create ticket"
}
```

Thread Retrieval

Use Slack API:

```
conversations.replies
```

Inputs:

- channel
- thread_ts

Retrieve:

- parent message
- replies

LLM Prompting Strategy

System Prompt Responsibilities

The LLM should:

- Identify operational issues
- Ignore irrelevant chatter
- Produce concise ticket titles
- Generate structured descriptions

- Infer likely priority
- Preserve important technical details

The LLM should NOT:

- Invent facts
- Overstate certainty
- Assume missing details

Suggested Structured Output

Use strict JSON output.

Example schema:

```
{
  "title": "string",
  "description": "string",
  "priority": "Low | Medium | High | Critical",
  "labels": ["string"],
  "summary": "string"
}
```

Suggested Internal Models

SlackThreadContext

```python
class SlackThreadContext:
    channel_id: str
    thread_ts: str
    messages: list[SlackMessage]
```

SlackMessage

```python
class SlackMessage:
    user: str
```

```
    text: str
    timestamp: str
```

TicketIntent

```
class TicketIntent:
    title: str
    description: str
    priority: str
    labels: list[str]
    summary: str
```

Jira Integration

MVP Requirements

Create:

- Title / summary
- Description
- Priority
- Labels

Optional:

- Assignee
- Components

Do NOT overcomplicate Jira mappings initially.

Error Handling

If Thread Context Missing

Respond:

```
Please use this command inside a Slack thread.
```

If LLM Fails

Respond:

```
I couldn't confidently generate a ticket from this discussion.
Please try again with more context.
```

If Jira Creation Fails

Respond:

```
Failed to create Jira ticket.
Please try again later.
```

Security / Privacy

MVP Assumptions

- Only analyze explicitly invoked threads
- Never scan channels automatically
- Never process messages without user invocation
- Keep context scoped and explainable

This is important for:

- User trust
- Enterprise adoption
- Privacy concerns

Future Roadmap (NOT MVP)

Potential future enhancements:

- Channel-wide context selection
- Time-window context analysis
- Incident clustering
- Duplicate ticket detection

- Auto-suggested tickets
- Service ownership inference
- Jira auto-assignment
- Operational memory systems
- Cross-thread incident reconstruction
- Postmortem generation

These should NOT be implemented during MVP phase.

Success Criteria

The MVP is successful if users say:

- "This saves me time."
- "The generated tickets are useful."
- "This fits naturally into Slack workflows."
- "I would use this again."

The MVP does NOT need:

- perfect AI
- autonomous reasoning
- advanced operational intelligence

The goal is workflow validation, not full automation.

Recommended Build Priority

Phase 1

- Slack app mentions
- Thread retrieval
- Basic LLM summarization
- Jira ticket creation

Phase 2

- Draft confirmation UI
- Better formatting
- Improved prompts
- Error handling

Phase 3

- Customer feedback iteration
- Workflow refinements
- Feature prioritization

Key Product Philosophy

This product should feel like:

- a lightweight operational assistant
- not an autonomous agent

Human review and explicit invocation are core design principles for the MVP.